Hyperion Terminal Transformer: A Mathematical Architecture and Evaluation Protocol

Hyperion Research

Independent research project

Version 2.0 — 8 August 2026 — architecture HTT-1.5B-r0

This is a prospective design study and preregistered evaluation protocol. No Hyperion Terminal Transformer checkpoint has been trained or evaluated. Numerical values in this paper are either algebraic consequences of the proposed dimensions, explicit engineering targets, or results attributed to prior work. They are not Hyperion benchmark results.

Abstract

The Hyperion Terminal Transformer (HTT) is a proposed narrow decoder-only language model for two artifact families: inert Terminal trading-strategy source and declarative Terminal visualizer specifications. It is neither a general assistant nor an autonomous trading system. This paper specifies the architecture as a falsifiable mathematical object, derives its parameter, operation, and memory budgets from first principles, and defines the training, retrieval, constrained-decoding, verification, calibration, and statistical protocols required to evaluate it honestly.

The reference configuration contains 32 transformer blocks, hidden width 2,048, 16 query heads, four key/value heads, head width 128, SwiGLU width 5,632, a 32,768-token vocabulary, and a maximum context of 32,768 tokens. A three-local/one-global attention schedule is repeated eight times. The core model plus two deterministic rank-64 task lanes contains approximately 1.523 billion stored parameters; the optional parser-state table raises this to approximately 1.524 billion. At 32K context and batch size one, the proposed hybrid schedule requires exactly 0.6875 GiB of BF16 key/value cache under the stated assumptions. During causal prefill, it reduces attention score positions by 57.42 percent relative to an all-global schedule; during steady-state decoding, the corresponding per-query reduction is 65.625 percent. These are derived quantities, not measured latency claims.

Accuracy is defined as sealed functional correctness after hard deterministic gates and calibrated request-level selection. The protocol reports unconditional accepted yield, selective accuracy, coverage, compile-at-one, behavioral pass rates, visual fidelity, latency, memory, and energy. It separates syntax from semantics, changing private facts from stable skills, and model evidence from release authority. The result is a detailed research baseline whose central claims can be rejected by controlled ablations and future sealed experiments.

1 Introduction and research contributions

Terminal artifact authoring is a constrained program-synthesis problem. A useful system must obey exact software-development-kit (SDK) and schema contracts, remain inside a restricted capability surface, cite the evidence on which it depends, and emit an artifact that deterministic tools can compile, render, execute in isolation, and test. A broad language model may carry billions of parameters devoted to unrelated knowledge while still failing these domain-specific obligations. The research question is therefore not "How small can a chatbot be?" but rather:

> What is the smallest reproducible sequence model that can produce correct Terminal artifacts when current private facts, policy, verification, and release authority remain outside its weights?

The proposed answer is HTT-1.5B-r0, a dense 1.5B-class decoder with two deterministic task lanes, selective retrieval, grammar-constrained generation, compiler and renderer feedback, request-level calibration, and abstention. The paper makes five research contributions:

1. a dimensionally complete mathematical specification of the transformer, conditioning features, local/global schedule, and task adapters;

2. exact parameter and key/value-cache counts plus explicit operation-count approximations;
3. a domain-limited training program whose losses are normalized by task and artifact region;
4. an authority-preserving inference protocol in which hard gates cannot be overridden by a learned score; and
5. a sealed evaluation design that measures accuracy after selection, calibration, latency, memory, and quantization under paired statistical tests.

The proposal authorizes nothing by itself. Full pretraining, customer-data use, deployment, strategy installation, order execution, or a change to the pinned `public/` shared-core boundary requires a separate governance decision.

1.1 Evidence classes

Every material statement is one of four kinds:

Implemented boundary.
Behavior represented by current Hyperion contracts or repository documentation.
Source-reported result.
A finding attributed to cited prior work and not reproduced here.
Derived quantity.
Arithmetic that follows from declared dimensions and assumptions.
Research hypothesis.
A proposed choice that must survive a preregistered ablation and sealed evaluation.

This separation prevents a cited model's benchmark score or a calculated memory lower bound from becoming an implied Hyperion result.

2 Scope, authority, and threat boundary

2.1 Allowed purposes

The coordinator, not the model, selects exactly one purpose:

- `STRATEGY`: author or repair inert C# or DAXQ strategy artifacts inside the restricted Terminal strategy surface; or
- `VISUALIZER`: author or repair a declarative visualizer specification that a deterministic compiler may later translate into an allowlisted renderer.

The model may explain fields and diagnostics of the artifact being produced. It is not designed for news, biography, open-domain programming, discretionary trade calls, portfolio allocation, broker operations, credential handling, arbitrary desktop automation, or unrestricted web access.

The current strategy result is a `CoordinatorStrategySourceArtifact`. It binds objectives, evidence identifiers, role outputs, exact file hashes, and source files. The future visualizer shape, `hyperion-terminal-visualizer/v1`, is deliberately non-executable: it contains allowlisted data sources, transformations, panes, series, axes, legends, interactions, accessibility labels, and bounded update behavior, but no network, filesystem, reflection, process, package, native-code, broker, or credential field.

2.2 What "Terminal-only knowledge" can mean

No executable code model can be proven to contain literally zero outside knowledge: C#, JSON, mathematics, and natural language already contain general regularities. The defensible operational claim is narrower:

1. admitted weight-training records are restricted by rights, domain, and provenance;
2. mutable private APIs and conventions are supplied through revocable, versioned retrieval;

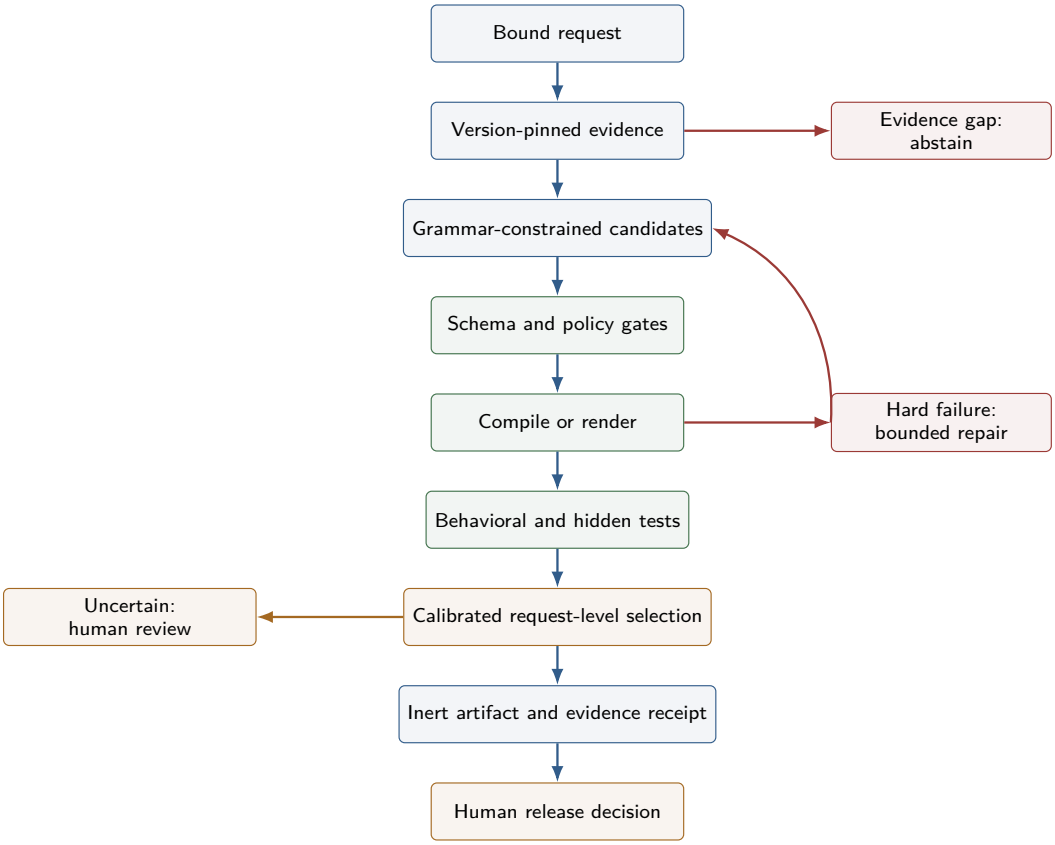


Figure 1: **Authority-preserving artifact lifecycle.** Generation can repair or abstain, but a learned score never bypasses a hard deterministic gate.

3. out-of-scope requests are rejected before generation or mapped to a refusal artifact;
4. grammar masks expose only the allowed artifact language;
5. compilers, renderers, tests, and policy engines determine executable validity; and
6. the served model has no direct network, filesystem, package, broker, or release authority.

3 Notation and abbreviations

3.1 Mathematical conventions

Vectors are bold lowercase, matrices are bold uppercase, and all token representations are row vectors unless an equation explicitly transposes them. Natural logarithms are used throughout. A multiply-add is counted as two floating-point operations in operation estimates. Parameter counts exclude biases unless stated. Memory uses bytes, decimal GB (10^9 bytes), and binary GiB (2^{30} bytes) distinctly. Context length and evaluation sample size use different symbols.

Table 1: Principal mathematical symbols.

Symbol	Domain or shape	Meaning
$\lvert\mathcal{V}\rvert$	$\mathbb{N}$	tokenizer vocabulary size
N_{ctx}	$\mathbb{N}$	current context length in tokens
N_{eval}	$\mathbb{N}$	number of evaluation requests
d	$\mathbb{N}$	hidden width
d_h	$\mathbb{N}$	attention-head width
d_{ff}	$\mathbb{N}$	SwiGLU intermediate width

Symbol	Domain or shape	Meaning
n_{layer}	$\mathbb{N}$	decoder-block count
h_q, h_{kv}	$\mathbb{N}$	query-head and key/value-head counts
W	$\mathbb{N}$	local attention window, including the current token
$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$	matrices	attention queries, keys, and values
ℓ	{strategy, visualizer}	server-selected task lane
r_a	$\mathbb{N}$	task-adapter rank
K_{cand}	$\mathbb{N}$	candidates generated per request
τ	$[0, 1]$	post-selection acceptance threshold
$\mathcal{L}$	$\mathbb{R}_{\geq 0}$	a loss; subscripts identify the objective
P	$\mathbb{N}$	parameter count
M	bytes	memory quantity
B	$\mathbb{N}$	inference batch size

3.2 Abbreviation glossary

Code typography is reserved for literal program identifiers such as `OnBar`, `schemaVersion`, or `STRATEGY`. Mathematical and engineering abbreviations are ordinary small-cap text.

Table 2: Abbreviations used in the paper.

Abbreviation	Meaning
HTT	Hyperion Terminal Transformer
LLM	large language model
NTP/MTP	next-token prediction / multi-token prediction
MHA/MQA/GQA	multi-head / multi-query / grouped-query attention
KV/QK	key–value / query–key
RMSNORM/ROPE	root-mean-square normalization / rotary position encoding
FFN/GLU/SWIGLU	feed-forward network / gated linear unit / SiLU-gated GLU
FIM	fill in the middle
BPE	byte-pair encoding
RAG	retrieval-augmented generation
SFT/DPO	supervised fine-tuning / direct preference optimization
RL/RLVR/GRPO/RLOO	reinforcement learning / reinforcement learning with verifiable rewards / group-relative policy optimization / reinforce leave-one-out
KL/NLL/ECE	Kullback–Leibler divergence / negative log likelihood / expected calibration error
IR/AST/IL	intermediate representation / abstract syntax tree / intermediate language
SDK/API/DSL	software development kit / application programming interface / domain-specific language
BF16/FP8/INT8/INT4	numeric storage or arithmetic formats
GPU/CUDA/IO	graphics processor / NVIDIA compute platform / input–output
AWQ/GPTQ	activation-aware weight quantization / post-training weight quantization
MOE/BLT	mixture of experts / byte latent transformer
DAXQ	Hyperion's restricted numeric strategy language; no expansion is assumed

4 Formal task and artifact model

Let a request be

$$\mathcal{R} = (u, \ell, s, v, \mathcal{E}, C, \mathcal{B}), \tag{1}$$

where u is the bounded user objective, ℓ is the server-selected task lane, s is the output schema identity, v is the Terminal/SDK version, $\mathcal{E}$ is an authorized evidence set, C is the deterministic capability policy, and $\mathcal{B}$ contains token, tool, time, and candidate budgets. The model emits a candidate artifact y and non-authoritative diagnostics. The coordinator recomputes all hashes, identities, and authority fields.

Define hard predicates $g_m(y, \mathcal{R}) \in \{0, 1\}$ for schema validity, policy, compilation or rendering, resource limits, prohibited capabilities, and required behavioral tests. Hard feasibility is

$$H(y, \mathcal{R}) = \prod_{m=1}^{M_g} g_m(y, \mathcal{R}). \tag{2}$$

No learned score can change a zero in this product into a one. The feasible set for request j is

$$\mathcal{F}_j = \{, i \in \{1, \dots, K_{\text{cand}}\} : H(y_{ji}, \mathcal{R}_j) = 1 \}. \tag{3}$$

If $\mathcal{F}_j = \varnothing$, the system repairs within a fixed budget or abstains. Human release is a later decision outside the model.

5 Transformer foundations and complete block equations

5.1 Token representation and immutable conditioning

For tokens $t_1, \dots, t_{N_{\text{ctx}}} \in \mathcal{V}$, let $\boldsymbol{E}_{\text{tok}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ be the tied input/output embedding. The input to position i is

$$\boldsymbol{x}_i^{(0)} = \boldsymbol{E}_{\text{tok}}[t_i] + \boldsymbol{e}_{r_i}^{\text{role}} + \boldsymbol{e}_{\ell}^{\text{task}} + \boldsymbol{e}_{s}^{\text{schema}} + \delta_{\text{parser}} \boldsymbol{E}_{\text{grammar}}[g_i], \tag{4}$$

$$\boldsymbol{X}^{(0)} = [\boldsymbol{x}_1^{(0)}; \dots; \boldsymbol{x}_{N_{\text{ctx}}}^{(0)}] \in \mathbb{R}^{N_{\text{ctx}} \times d}. \tag{5}$$

Here $\delta_{\text{parser}} \in \{0, 1\}$ controls the optional parser-state feature and g_i is computed only from the already emitted prefix, so it cannot leak future tokens. Absolute position is not added to equation (4); ROPE acts on queries and keys.

5.2 Why dot-product attention is scaled

For one head, take

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V, \tag{6}$$

with $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N_{\text{ctx}} \times d_h}$. Suppose, only for the variance calculation, that components q_m and k_m are independent, mean zero, and have variance σ^2. Then

$$\mathbb{E}[\boldsymbol{q}^\top \boldsymbol{k}] = \sum_{m=1}^{d_h} \mathbb{E}[q_m k_m] = 0, \tag{7}$$

$$\text{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = \sum_{m=1}^{d_h} \text{Var}(q_m k_m) = d_h \sigma^4. \tag{8}$$

Without scaling, score variance grows linearly with head width and pushes the softmax toward saturated probabilities. Dividing by $\sqrt{d_h}$ yields

$$\text{Var}\left(\frac{\boldsymbol{q}^\top \boldsymbol{k}}{\sqrt{d_h}}\right) = \sigma^4, \tag{9}$$

which is independent of d_h under the stated approximation. The masked attention matrix is

$$\boldsymbol{S} = \frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_h}} + \boldsymbol{M}, \tag{10}$$

$$M_{ij} = \begin{cases} 0, & j \le i \text{ and } j \text{ is inside the layer's window,} \\ -\infty, & \text{otherwise,} \end{cases} \tag{11}$$

$$\mathrm{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}_{\mathrm{row}}(\boldsymbol{S})\boldsymbol{V}. \tag{12}$$

The mathematical mask is $-\infty$; an implementation uses the most negative safe value for the chosen kernel and dtype. Exact-attention kernels may reorder floating-point operations while remaining algorithmically equivalent to equation (12); "exact" does not mean bitwise identical.

5.3 Multi-head and grouped-query attention

With h_q query heads, the output is

$$\mathrm{MHA}(\boldsymbol{X}) = \mathrm{Concat}(\boldsymbol{O}_1, \ldots, \boldsymbol{O}_{h_q})\boldsymbol{W}_O, \qquad \boldsymbol{W}_O \in \mathbb{R}^{h_q d_h \times d}. \tag{13}$$

Ordinary multi-head attention has h_q key and value heads. Multi-query attention has one. Grouped-query attention (GQA) chooses $h_{\mathrm{kv}} \mid h_q$ and group size

$$g = \frac{h_q}{h_{\mathrm{kv}}}, \qquad \kappa(a) = 1 + \left\lfloor \frac{a-1}{g} \right\rfloor, \tag{14}$$

so query head a uses key/value head $\kappa(a)$. For bias-free projections,

$$P_Q = d(h_q d_h), \qquad P_K = d(h_{\mathrm{kv}} d_h), \tag{15}$$

$$P_V = d(h_{\mathrm{kv}} d_h), \qquad P_O = (h_q d_h)d. \tag{16}$$

Because $h_q d_h = d$ in HTT,

$$P_{\mathrm{attn}} = 2d^2 + 2d h_{\mathrm{kv}} d_h. \tag{17}$$

For $d = 2{,}048$, $h_q = 16$, $h_{\mathrm{kv}} = 4$, and $d_h = 128$, this is

$$P_{\mathrm{attn}} = 2(2{,}048)^2 + 2(2{,}048)(4)(128) \tag{18}$$

$$= 8{,}388{,}608 + 2{,}097{,}152 \tag{19}$$

$$= 10{,}485{,}760 \quad \text{parameters per layer.} \tag{20}$$

5.4 Rotary position encoding

Let $m \in \{0, \ldots, d_h/2 - 1\}$ index coordinate pairs and define frequency

$$\omega_m = b^{-2m/d_h}, \tag{21}$$

where b is the selected ROPE base. At position p, pair m is rotated by

$$\boldsymbol{R}_m(p) = \begin{bmatrix} \cos(p\omega_m) & -\sin(p\omega_m) \\ \sin(p\omega_m) & \cos(p\omega_m) \end{bmatrix}. \tag{22}$$

The full $\boldsymbol{R}(p)$ is block diagonal with the $d_h/2$ pair matrices. Orthogonality and angle addition give

$$\boldsymbol{R}_m(p)^\top \boldsymbol{R}_m(r) = \boldsymbol{R}_m(-p)\boldsymbol{R}_m(r) \tag{23}$$

$$= \boldsymbol{R}_m(r - p). \tag{24}$$

Therefore the rotated dot product is

$$(\boldsymbol{R}(p)\boldsymbol{q})^\top (\boldsymbol{R}(r)\boldsymbol{k}) = \boldsymbol{q}^\top \boldsymbol{R}(r - p)\boldsymbol{k}, \tag{25}$$

which depends on relative displacement $r - p$ while retaining content-dependent $\boldsymbol{q}$ and $\boldsymbol{k}$ [4].

5.5 RMSNorm, QK-Norm, and the pre-normalized residual path

For $\boldsymbol{x} \in \mathbb{R}^d$, learned gain $\boldsymbol{\gamma} \in \mathbb{R}^d$, and $\epsilon > 0$, define

$$\rho(\boldsymbol{x}) = \sqrt{\frac{1}{d}\boldsymbol{x}^\top \boldsymbol{x} + \epsilon}, \tag{26}$$

$$\mathrm{RMSNorm}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{x}}{\rho(\boldsymbol{x})}. \tag{27}$$

The Jacobian exposes both the direct scaling path and the rank-one normalization correction:

$$\frac{\partial\, \mathrm{RMSNorm}(\boldsymbol{x})}{\partial \boldsymbol{x}} = \mathrm{diag}(\boldsymbol{\gamma}) \left[\frac{\boldsymbol{I}}{\rho(\boldsymbol{x})} - \frac{\boldsymbol{x}\boldsymbol{x}^\top}{d\, \rho(\boldsymbol{x})^3} \right]. \tag{28}$$

Ignoring ϵ, multiplying $\boldsymbol{x}$ by a positive scalar cancels between numerator and denominator, explaining the normalization's approximate scale invariance [2].

HTT additionally normalizes each query and key head across its d_h coordinates before ROPE:

$$\widehat{\boldsymbol{q}}_{ia} = \mathrm{RMSNorm}_q(\boldsymbol{q}_{ia}), \qquad \widehat{\boldsymbol{k}}_{ib} = \mathrm{RMSNorm}_k(\boldsymbol{k}_{ib}). \tag{29}$$

The reference count assumes one d_h-element gain shared across query heads and one shared across key heads, adding $2d_h = 256$ parameters per block. Per-head gains are a different, larger model and must be counted separately.

For block function F_l, pre-normalization uses

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + F_l(\mathrm{RMSNorm}(\boldsymbol{x}_l)). \tag{30}$$

Its local Jacobian has an explicit identity term,

$$\frac{\partial \boldsymbol{x}_{l+1}}{\partial \boldsymbol{x}_l} = \boldsymbol{I} + \boldsymbol{J}_{F_l} \boldsymbol{J}_{\mathrm{RMS},l}, \tag{31}$$

so the residual stream contains a depth-independent identity route. This does not guarantee stable training, but it avoids forcing every gradient through a terminal normalization transform.

5.6 SwiGLU and the width-matching derivation

For row input $\boldsymbol{x} \in \mathbb{R}^d$, define

$$\boldsymbol{u} = \boldsymbol{x}\boldsymbol{W}_u, \qquad \boldsymbol{g} = \boldsymbol{x}\boldsymbol{W}_g, \tag{32}$$

$$\mathrm{SiLU}(z) = z\,\mathrm{sigmoid}(z), \qquad \mathrm{SwiGLU}(\boldsymbol{x}) = [\mathrm{SiLU}(\boldsymbol{g}) \odot \boldsymbol{u}]\, \boldsymbol{W}_d, \tag{33}$$

where $\boldsymbol{W}_u, \boldsymbol{W}_g \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$ and $\boldsymbol{W}_d \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$. Its bias-free parameter count is

$$P_{\mathrm{ffn}} = 3 d d_{\mathrm{ff}}. \tag{34}$$

A conventional two-matrix feed-forward network of width $4d$ contains $d(4d) + (4d)d = 8d^2$ parameters. Matching the SwiGLU count gives

$$3 d d_{\mathrm{ff}} \approx 8d^2, \tag{35}$$

$$d_{\mathrm{ff}} \approx \frac{8d}{3}. \tag{36}$$

At $d = 2{,}048$, the exact rule yields $5{,}461\frac{1}{3}$. The proposed tensor-friendly value 5,632 is $170\frac{2}{3}$ units, or 3.125 percent, larger. Consequently

$$P_{\mathrm{ffn}} = 3(2{,}048)(5{,}632) = 34{,}603{,}008. \tag{37}$$

SwiGLU is adopted as a candidate supported by prior matched-compute language-model results, not as a guaranteed HTT improvement [3].

Table 3: Reference architecture. Every value is a hypothesis until an ablation establishes it.

Component	Proposed value	Required comparison
Decoder blocks	32	24 and 36 blocks at matched active parameters
Hidden width d	2,048	1,792 and 2,304
Query / key-value heads	16 / 4	$h_{\mathrm{kv}} \in \{1, 2, 8, 16\}$
Head width d_h	128	64 and 128 at matched hidden width
SwiGLU width d_{ff}	5,632	5,504 and 6,144
Vocabulary $\lvert\mathcal{V}\rvert$	32,768	16K, 48K, and inherited large tokenizer
Maximum context	32,768	8K and 16K operating points
Local window W	4,096	2K, 8K, and all-global
Global schedule	every fourth block	every sixth block and all-global
Task lanes	rank 64 in top 12 blocks	shared trunk and separate models
Training baseline	BF16	FP8 only after numerical parity

5.7 Complete decoder block

For layer l, suppressing dropout because the serving graph is deterministic,

$$\boldsymbol{A}_l = \mathrm{GQA}_l\big(\mathrm{RMSNorm}_{l,1}(\boldsymbol{X}_l); \boldsymbol{M}_l\big), \tag{38}$$

$$\widetilde{\boldsymbol{X}}_l = \boldsymbol{X}_l + \boldsymbol{A}_l + \mathcal{A}^{\mathrm{attn}}_{l,\ell}(\boldsymbol{X}_l), \tag{39}$$

$$\boldsymbol{F}_l = \mathrm{SwiGLU}_l\Big(\mathrm{RMSNorm}_{l,2}(\widetilde{\boldsymbol{X}}_l)\Big), \tag{40}$$

$$\boldsymbol{X}_{l+1} = \widetilde{\boldsymbol{X}}_l + \boldsymbol{F}_l + \mathcal{A}^{\mathrm{ffn}}_{l,\ell}(\widetilde{\boldsymbol{X}}_l). \tag{41}$$

The task-adapter maps $\mathcal{A}$ are zero outside the top 12 blocks. A final RMSNorm produces $\boldsymbol{H}$, and tied logits are

$$\boldsymbol{Z} = \boldsymbol{H}\boldsymbol{E}_{\mathrm{tok}}^{\top}. \tag{42}$$

6 Proposed HTT-1.5B-r0 architecture

6.1 Local/global attention schedule

Blocks follow three local layers followed by one global layer, repeated eight times. A local query at position i can attend to keys

$$j \in \{\max(1, i - W + 1), \ldots, i\}. \tag{43}$$

Because the window includes the current token, one local layer expands the maximum causal receptive span by $W - 1$. Ignoring optimization and attenuation, after k consecutive local layers the exact maximum span is

$$R_k = 1 + k(W - 1). \tag{44}$$

A global layer directly connects every position to the full causal prefix. The 3:1 schedule is more global than the 5:1 pattern reported for Gemma 3 because cross-file references and compiler repair may need frequent long-range exchange; that rationale is a hypothesis and the 5:1 schedule remains a mandatory challenger [8, 36].

6.2 Exact causal score counts

For one all-global causal layer with context $N = N_{\mathrm{ctx}}$, the number of unmasked query–key positions is the triangular sum

$$S_G(N) = \sum_{i=1}^{N} i = \frac{N(N+1)}{2}. \tag{45}$$

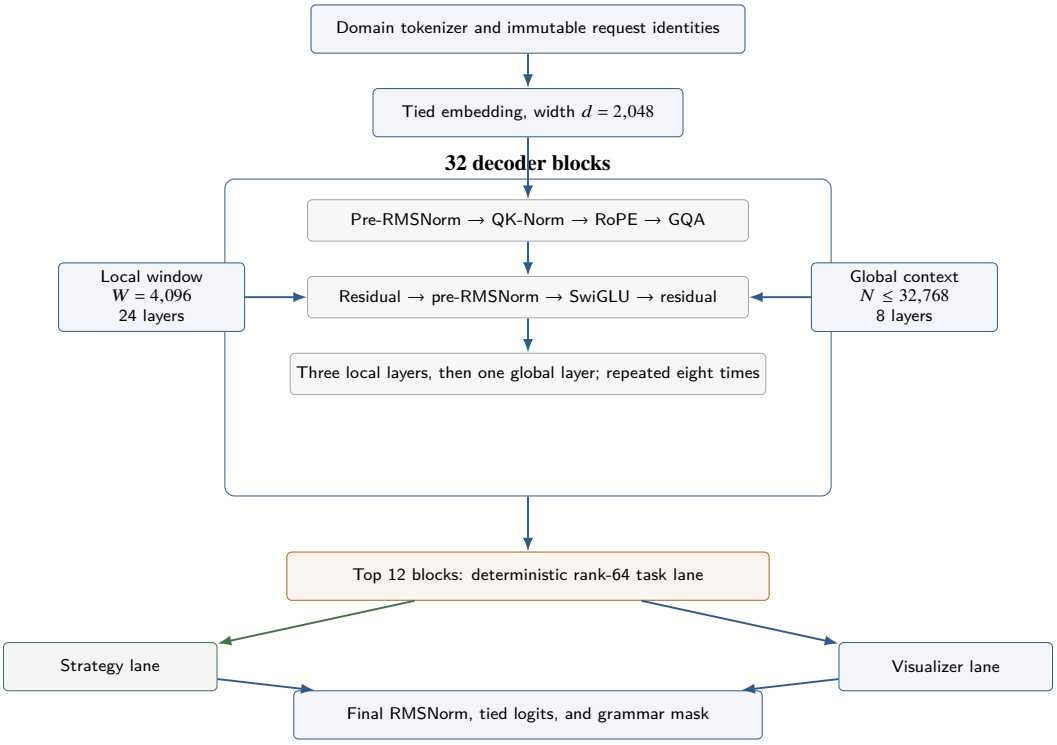


Figure 2: **HTT-1.5B-r0 computational graph.** The task lane is chosen by the coordinator, never by a learned router. Grammar masking and deterministic tools remain outside the backbone.

For a local layer with window $W \le N$,

$$S_L(N,W) = \sum_{i=1}^{N} \min(i,W) \tag{46}$$

$$= \sum_{i=1}^{W} i + \sum_{i=W+1}^{N} W \tag{47}$$

$$= \frac{W(W+1)}{2} + (N-W)W \tag{48}$$

$$= NW - \frac{W(W-1)}{2}. \tag{49}$$

With $L_G = 8$ global and $L_L = 24$ local blocks, the causal-prefill score ratio relative to 32 all-global blocks is

$$R_{\text{prefill}}(N,W) = \frac{L_G S_G(N) + L_L S_L(N,W)}{(L_G + L_L) S_G(N)}. \tag{50}$$

At $N = 32{,}768$ and $W = 4{,}096$,

$$S_G = 536{,}887{,}296, \tag{51}$$

$$S_L = 125{,}831{,}168, \tag{52}$$

$$S_{\text{hybrid}} = 8S_G + 24S_L = 7{,}315{,}046{,}400, \tag{53}$$

$$S_{\text{all global}} = 32S_G = 17{,}180{,}393{,}472, \tag{54}$$

$$R_{\text{prefill}} = 0.4257787467. \tag{55}$$

Thus the exact causal score-position reduction is

$$1 - R_{\text{prefill}} = 0.5742212533, \tag{56}$$

or 57.42 percent. The earlier rectangular approximation $[8N + 24W]/(32N) = 0.34375$ describes one steady-state decode query at full context, not causal prefill. For that decode regime, the score-

position reduction is 65.625 percent. Keeping these regimes separate prevents an inference-only ratio from being reported as a training-compute result.

6.3 Deterministic low-rank task lanes

For row vector $\boldsymbol{x} \in \mathbb{R}^d$, adapter rank $r_a = 64$, and lane ℓ, define

$$\mathcal{A}_{l,\ell}(\boldsymbol{x}) = \mathrm{SiLU}(\boldsymbol{x}\boldsymbol{A}_{l,\ell})\boldsymbol{B}_{l,\ell}, \quad \boldsymbol{A}_{l,\ell} \in \mathbb{R}^{d\times r_a}, \quad \boldsymbol{B}_{l,\ell} \in \mathbb{R}^{r_a\times d}. \tag{57}$$

There are two adapter positions in each of the top 12 blocks. With no biases and no learned scalar, one lane contains

$$\begin{aligned} P_{\text{lane}} &= 12 \text{ blocks} \times 2 \text{ positions} \times (2dr_a) && (58)\\ &= 12 \times 2 \times 2 \times 2{,}048 \times 64 && (59)\\ &= 6{,}291{,}456. && (60) \end{aligned}$$

Initializing every $\boldsymbol{B}_{l,\ell} = \boldsymbol{0}$ makes the initial network exactly equal to the shared trunk while allowing gradients into $\boldsymbol{B}$. Merely saying "near zero" would not establish exact equality. A learned scalar per adapter would add 24 parameters per lane and must be declared in the executable manifest.

The lane is an inductive bias, not an authority mechanism. It cannot select a schema, retrieve evidence, grant a capability, or release an artifact. A learned mixture-of-experts router is deferred until the dense baseline is reproducible because sparse routing introduces expert balance, communication, quantization, storage, and rollback questions [49, 50].

7 Parameter-count derivation

This section derives every reference-graph count from matrix shapes. Let

$$|\mathcal{V}| = 32{,}768, \quad d = 2{,}048, \quad n_{\text{layer}} = 32, \quad h_q = 16, \quad h_{\text{kv}} = 4, \quad d_h = 128, \quad d_{\text{ff}} = 5{,}632. \tag{61}$$

7.1 Embedding, block, and identity parameters

The tied token embedding/output matrix contributes

$$P_{\text{embed}} = |\mathcal{V}|d = 32{,}768 \times 2{,}048 = 67{,}108{,}864. \tag{62}$$

From equations (20) and (37), plus two d-element RMSNorm gains and two shared d_h-element QK-Norm gains,

$$\begin{aligned} P_{\text{block}} &= P_{\text{attn}} + P_{\text{ffn}} + 2d + 2d_h && (63)\\ &= 10{,}485{,}760 + 34{,}603{,}008 + 4{,}096 + 256 && (64)\\ &= 45{,}093{,}120, && (65)\\ P_{\text{blocks}} &= 32P_{\text{block}} = 1{,}442{,}979{,}840. && (66) \end{aligned}$$

The final RMSNorm adds $d = 2{,}048$ parameters.

Identity embeddings must be auditable rather than represented by an unexplained approximation. The reference budget reserves 16 identity rows: eight coordinator roles, two task lanes, four schema families, and two boundary states. Therefore

$$P_{\text{identity}} = d(8 + 2 + 4 + 2) = 2{,}048 \times 16 = 32{,}768. \tag{67}$$

Changing these cardinalities changes the count. The core shared base is

$$\begin{aligned} P_{\text{base}} &= P_{\text{embed}} + P_{\text{blocks}} + d + P_{\text{identity}} && (68)\\ &= 67{,}108{,}864 + 1{,}442{,}979{,}840 + 2{,}048 + 32{,}768 && (69)\\ &= 1{,}510{,}123{,}520. && (70) \end{aligned}$$

Table 4: Parameter ledger. Counts assume bias-free linear layers, shared QK-Norm gains, fixed adapter scales, and tied input/output embeddings.

Component	Parameters	Status
Tied token embedding	67,108,864	always stored and active
32 transformer blocks	1,442,979,840	always stored and active
Final RMSNorm	2,048	always stored and active
Identity embeddings	32,768	declared 16-row budget
Strategy task lane	6,291,456	stored; active only for strategy
Visualizer task lane	6,291,456	stored; active only for visualizer
Optional parser table	1,048,576	ablation, not required for grammar masking
Core stored total	1,522,706,432	both lanes, no parser table
Parser-enabled stored total	1,523,755,008	both lanes and parser table
Parser-enabled active total	1,517,463,552	one lane and parser table

7.2 Stored and active totals

Two task lanes add

$$P_{\text{lanes}} = 2P_{\text{lane}} = 12{,}582{,}912. \tag{71}$$

The core stored total and one-lane active total are

$$P_{\text{stored,core}} = P_{\text{base}} + P_{\text{lanes}} = 1{,}522{,}706{,}432, \tag{72}$$

$$P_{\text{active,core}} = P_{\text{base}} + P_{\text{lane}} = 1{,}516{,}414{,}976. \tag{73}$$

The optional parser-state table has 512 states:

$$P_{\text{parser}} = 512d = 1{,}048{,}576. \tag{74}$$

Consequently

$$P_{\text{stored,parser}} = 1{,}523{,}755{,}008, \tag{75}$$

$$P_{\text{active,parser}} = 1{,}517{,}463{,}552. \tag{76}$$

The executable model graph is authoritative. Biases, per-head QK gains, padded vocabulary rows, learned adapter scales, or a different identity cardinality must appear in a signed build manifest.

7.3 Vocabulary comparison

Replacing a tied 151,646-token embedding with the proposed 32,768-token embedding saves

$$\Delta P_{\text{vocab}} = (151{,}646 - 32{,}768)d \tag{77}$$

$$= 118{,}878 \times 2{,}048 \tag{78}$$

$$= 243{,}462{,}144 \tag{79}$$

parameters in one tied matrix. Untied input and output matrices would double the saving. This is not automatically a latency win because a smaller vocabulary can tokenize the same artifact into more steps; the tokenizer study must jointly measure embedding size, tokens per character, exact literal copying, compile rate, and end-to-end latency [11, 12].

8 Operation-count and memory derivations

8.1 Forward matrix-multiplication operations

Count one multiply-add as two floating-point operations. For sequence length N, attention projections in one block cost

$$C_{\text{proj}} = 4Nd^2 + 4Nd\, h_{\text{kv}} d_h, \tag{80}$$

where the first term covers query and output projections and the second covers key and value projections. Three SwiGLU matrices cost

$$C_{\text{ffn}} = 6Nddff. \tag{81}$$

For a layer with S unmasked score positions, $\boldsymbol{QK}^\top$ and attention–value multiplication together cost approximately

$$C_{\text{scores}} = 4h_q d_h S = 4dS. \tag{82}$$

Thus the 32-block hybrid forward matmul count is

$$C_{\text{hybrid}} = 32(C_{\text{proj}} + C_{\text{ffn}}) + 4d\,[8S_G(N) + 24S_L(N, W)]. \tag{83}$$

At $N = 32{,}768$ and $W = 4{,}096$, the declared dimensions give approximately

$$C_{\text{linear,one block}} = 2.95494 \times 10^{12}, \tag{84}$$

$$C_{\text{hybrid}} = 1.54483 \times 10^{14}, \tag{85}$$

$$C_{\text{all global}} = 2.352998 \times 10^{14}. \tag{86}$$

The ratio is 0.6565, an approximate 34.35 percent reduction in these forward matmuls. This is smaller than the score-position reduction because projections and SwiGLU are unchanged. It excludes softmax, normalization, activation functions, memory traffic, padding, and fixed kernel overhead; only profiler measurements can establish latency or energy.

8.2 Weight-storage lower bounds

For P parameters stored at b_w bytes each,

$$M_{\text{weights}} = Pb_w. \tag{87}$$

Using the parser-enabled stored total $P = 1{,}523{,}755{,}008$:

Table 5: Weight-only storage lower bounds for the parser-enabled stored model. Quantization scales, metadata, alignment, runtime workspaces, and duplicate buffers are excluded.

Format	Decimal GB	GiB	Interpretation
BF16, 2 bytes	3.0475	2.8382	reference inference/checkpoint lower bound
INT8, 1 byte	1.5238	1.4191	requires exact-artifact parity evaluation
INT4, 0.5 byte	0.7619	0.7096	scales and packing overhead excluded

Training state is much larger. A common mixed-precision Adam profile with BF16 weights (2 bytes), BF16 gradients (2), FP32 master weights (4), and two FP32 moments (8) uses approximately 16 bytes per parameter before activations and allocator overhead:

$$M_{\text{Adam,state}} \approx 16P = 24.38\ \text{GB} = 22.71\ \text{GiB}. \tag{88}$$

Different optimizers, gradient dtypes, sharding, and offload policies change this number; therefore a weight-only fit never establishes a training-hardware fit.

8.3 Key/value-cache memory

Each cached token stores one key and one value for every key/value head. With batch size B, layer count L, live tokens N, h_{kv} heads, head width d_h, and b_{kv} bytes per element,

$$M_{\text{KV}} = 2BLNh_{\text{kv}}d_h b_{\text{kv}}. \tag{89}$$

For L_G global and L_L rolling-local layers,

$$M_{\text{KV,hybrid}} = 2Bh_{\text{kv}}d_h b_{\text{kv}}\,[L_G N + L_L \min(N, W)]\,. \tag{90}$$

At $B = 1$, $h_{\text{kv}} = 4$, $d_h = 128$, BF16 $b_{\text{kv}} = 2$, $L_G = 8$, $L_L = 24$, $N = 32{,}768$, and $W = 4{,}096$,

$$M_{\text{KV,hybrid}} = 2(1)(4)(128)(2)\,[8(32{,}768) + 24(4{,}096)] \tag{91}$$
$$= 738{,}197{,}504 \text{ bytes} \tag{92}$$
$$= 0.6875 \text{ GiB}. \tag{93}$$

The comparison with 8 GiB for 16-head all-global attention contains two separate reductions: GQA reduces 8 GiB to 2 GiB, and local rolling caches reduce 2 GiB to 0.6875 GiB.

Table 6: Batch-one hybrid BF16 key/value cache.

Active context	Hybrid KV cache (GiB)
8,192	0.3125
16,384	0.4375
32,768	0.6875

The formula scales linearly with batch size and KV bytes. Paged allocation reduces fragmentation and enables non-contiguous cache management, but it does not change the mathematical bytes per live token [26].

8.4 Training-compute approximation

For a dense model, a parameter participates approximately once in the forward pass and twice in the backward pass. Each use is a multiply-add, so the parameterized portion is often approximated as

$$C_{\text{train,param}} \approx 6PT, \tag{94}$$

where T is the number of training tokens. The factor six is therefore not a law: it omits the non-parameterized attention term, embeddings with sparse lookup behavior, activation recomputation, padding, optimizer operations, and hardware utilization. For $P = P_{\text{active,parser}} = 1.517463552 \times 10^9$: At a sustained aggregate 1 PFLOP/s, the 100B-token parameterized estimate alone corresponds to

Table 7: Parameterized training-compute approximation.

Tokens T	$6PT$ floating-point operations
50×10^9	4.5524×10^{20}
100×10^9	9.1048×10^{20}
200×10^9	1.8210×10^{21}

approximately 10.54 days; at 100 TFLOP/s it is 105.4 days. These conversions are capacity-planning examples, not runtime predictions. Domain-specific scaling pilots must measure achieved utilization and add the exact hybrid-attention term before any full-run authorization [33, 34].

9 Tokenizer and sequence protocol

9.1 Lossless domain-tokenizer selection

Let $\mathcal{D}_j$ denote corpus domain j—for example strategy source, visualizer specifications, diagnostics, documentation, or numeric edge cases—and let $\alpha_j \geq 0$ with $\sum_j \alpha_j = 1$. A candidate tokenizer $\mathcal{T}$ must first satisfy the hard round-trip constraint

$$\text{decode}_{\mathcal{T}}(\text{encode}_{\mathcal{T}}(s)) = s \quad \forall s \text{ in the admitted byte corpus.} \tag{95}$$

Byte fallback makes an "invalid byte rate" redundant when implemented correctly; failures of equation (95) reject the tokenizer rather than becoming a soft objective.

For each domain define dimensionless, normalized measurements relative to a fixed reference tokenizer $\mathcal{T}_0$:

$$\widetilde{m}_{j,\text{length}}(\mathcal{T}) = \frac{\mathbb{E}_{s\sim\mathcal{D}_j}\left[|\mathcal{T}(s)|/|s|_{\text{byte}}\right]}{\mathbb{E}_{s\sim\mathcal{D}_j}\left[|\mathcal{T}_0(s)|/|s|_{\text{byte}}\right]}, \tag{96}$$

$$\widetilde{m}_{j,\text{id}}(\mathcal{T}) = \frac{\mathbb{E}[\text{identifier fragments per identifier under } \mathcal{T}]}{\mathbb{E}[\text{identifier fragments under } \mathcal{T}_0]}, \tag{97}$$

$$\widetilde{m}_{j,\text{num}}(\mathcal{T}) = \frac{\mathbb{E}[\text{numeric fragments per literal under } \mathcal{T}]}{\mathbb{E}[\text{numeric fragments under } \mathcal{T}_0]}. \tag{98}$$

When vocabulary size varies, include normalized embedding memory

$$\widetilde{m}_{\text{mem}}(\mathcal{T}) = \frac{|\mathcal{V}_{\mathcal{T}}|d}{|\mathcal{V}_{\mathcal{T}_0}|d}. \tag{99}$$

If every candidate has the same vocabulary size, this term is constant and must be removed. The selection objective is then

$$J_{\text{tok}}(\mathcal{T}) = \sum_j \alpha_j \left[\lambda_{\text{len}}\widetilde{m}_{j,\text{length}} + \lambda_{\text{id}}\widetilde{m}_{j,\text{id}} + \lambda_{\text{num}}\widetilde{m}_{j,\text{num}}\right] + \lambda_{\text{mem}}\widetilde{m}_{\text{mem}}, \tag{100}$$

with nonnegative preregistered weights. This selects among byte-fallback BPE or unigram candidates; it is not itself the merge-training objective. The final choice must also pass exact-literal copying, compile-at-one, and latency tests. Public code-model reports motivate repository and fill-in-the-middle markers, but HTT must establish their value in its own ablations [51, 53, 11].

9.2 Numeric and repository structure

Strategies contain signed decimals, dates, durations, enum-like constants, indicator periods, and quantities whose exact spelling matters. Tokenizer candidates are stratified by magnitude, sign, decimal precision, exponent notation, leading zeros, timestamp format, and out-of-distribution identifier shape. A candidate is rejected if token savings coincide with worse exact numeric copy or arithmetic behavior.

Reserved markers identify role, purpose, schema, evidence, repository, file, diagnostic, tool result, candidate boundary, refusal, and end of artifact. These are data delimiters, not authority tokens. The server constructs them and validates their ordering.

9.3 Packing, fill in the middle, and repair

Packing unrelated examples into one physical sequence is allowed only with a block-diagonal causal mask. If examples a and b occupy disjoint token sets I_a and I_b, the mask enforces

$$M_{ij} = -\infty \quad \text{when } i \in I_a,\ j \in I_b,\ a \neq b, \tag{101}$$

so no example conditions on another. Loss weights are zero on padding, prompt, retrieved evidence, and tool-result regions.

For a file decomposed into prefix p, missing span m, and suffix s, fill-in-the-middle training uses an explicit serialization such as

$$\langle\text{FIM_PREFIX}\rangle\, p\, \langle\text{FIM_SUFFIX}\rangle\, s\, \langle\text{FIM_MIDDLE}\rangle\, m. \tag{102}$$

Repair examples additionally bind the base artifact hash, exact diagnostic, allowed edit region, and expected post-repair checks. The model returns an inert full artifact or bounded patch; it never mutates a live file directly.

Table 8: Knowledge placement rule.

Knowledge	Location	Reason
C#, JSON, bounded mathematics, visual grammar	model weights	stable reusable composition
Current private APIs and conventions	signed retrieval pack	mutable, revocable, citable
Purpose, entitlement, budgets, policy	coordinator/policy service	authority cannot be delegated
Compilation and executable semantics	deterministic tools	machine-checkable behavior
Sealed answers and hidden cases	evaluation custody	prevents leakage and reward hacking

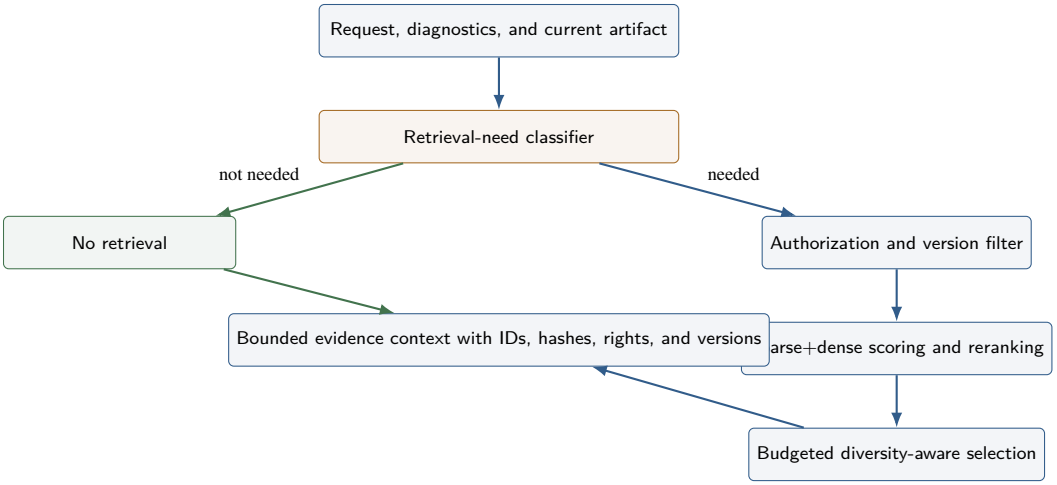


Figure 3: **Selective, version-aware retrieval.** Authorization and version filtering occurs before semantic ranking; selected evidence remains bounded and fully identified.

10 Knowledge placement and selective retrieval

Stable compositional skill belongs in weights; mutable or authoritative facts do not.

10.1 Filtering and hybrid relevance

For request representation $\boldsymbol{q}$ and evidence chunk j, define lexical score s_j^{lex}, dense similarity s_j^{dense}, cross-encoder score s_j^{cross}, version match v_j, and diagnostic-symbol match d_j. Standardize every continuous feature on development data:

$$\widetilde{s}_{jk} = \frac{s_{jk} - \mu_k}{\sigma_k + \epsilon}. \tag{103}$$

Only chunks passing rights, principal, tenant, availability-time, product-version, and source-integrity filters enter the set $\mathcal{A}(\mathcal{R})$. A relevance utility can then be

$$u_j = \beta_1 \widetilde{s}_j^{\text{lex}} + \beta_2 \widetilde{s}_j^{\text{dense}} + \beta_3 \widetilde{s}_j^{\text{cross}} + \beta_4 v_j + \beta_5 d_j. \tag{104}$$

The coefficients are fitted only on retrieval development labels. Iterative and selective repository retrieval are supported patterns, not proof for Terminal artifacts [16, 17, 18].

10.2 Budgeted diversity-aware selection

Let c_j be the token cost of chunk j, including delimiters and citation metadata, and let $z_j \in \{0,1\}$ indicate selection. A concrete budgeted objective is

$$\max_{z} \quad \sum_{j \in \mathcal{A}(\mathcal{R})} u_j z_j - \lambda_{\text{dup}} \sum_{j<k} \text{sim}(j,k) z_j z_k, \tag{105}$$

$$\text{subject to} \quad \sum_{j} c_j z_j \le B_{\text{evidence}}, \qquad z_j \in \{0,1\}. \tag{106}$$

The quadratic penalty discourages redundant chunks. Greedy marginal-gain selection is an approximation; the implementation must record the chosen method and deterministic tie breaks.

The complete context budget is

$$B_{\text{system}} + B_{\text{request}} + B_{\text{evidence}} + B_{\text{artifact}} + B_{\text{diagnostic}} + B_{\text{output}} + B_{\text{delimiters}} \le N_{\max}. \tag{107}$$

Long repositories are never dumped wholesale. Retrieval recall is evaluated independently from generation because a generator cannot legitimately compensate for missing authoritative evidence by inventing an API.

11 Corpus governance and data-mixture optimization

Every admitted record binds source identity, license and rights decision, acquisition time, content hash, product version, knowledge class, deduplication lineage, teacher identity for synthetic data, tool versions, and validation outcomes. The partitions are:

- *train*: admitted records visible to gradient updates;
- *development*: visible tasks for architecture and harness iteration;
- *calibration*: threshold fitting, never gradient training;
- *sealed*: independently held promotion comparison; and
- *future*: SDK- and time-forward cases acquired after the training cutoff.

Exact hashes, token MinHash, abstract-syntax-tree fingerprints, dependency graphs, prompts, and semantic clusters enforce split isolation. Deduplication is quality control as well as contamination control [20, 21].

Let k index corpus lanes and $\boldsymbol{\alpha}$ be mixture weights on the simplex Δ^{K-1}. Mixture selection is a bilevel problem:

$$\boldsymbol{\theta}^*(\boldsymbol{\alpha}) = \arg\min_{\boldsymbol{\theta}} \sum_{k=1}^{K} \alpha_k \mathcal{L}_k^{\text{train}}(\boldsymbol{\theta}), \tag{108}$$

$$\min_{\boldsymbol{\alpha} \in \Delta^{K-1}} \quad \max_{k} \widetilde{\mathcal{L}}_k^{\text{dev}}(\boldsymbol{\theta}^*(\boldsymbol{\alpha})), \tag{109}$$

$$\text{subject to} \quad \alpha_k \ge f_k \ge 0, qquad \sum_{k} f_k \le 1. \tag{110}$$

Development losses are normalized by a fixed reference so one intrinsically high-entropy lane does not dominate solely by scale. Floors f_k protect rare refusal, security, visual-accessibility, and numeric-edge lanes. Proxy optimization such as DoReMi is admissible only if it improves sealed functional slices rather than merely average language-model loss [19].

12 Training objectives and curriculum

12.1 Domain scaling pilots

Before a full run, fit domain scaling behavior across multiple model sizes and token budgets. A candidate phenomenological form is

$$\mathcal{L}(P, D) = \mathcal{L}_\infty + aP^{-\alpha} + bD^{-\beta}, \qquad a, b, \alpha, \beta > 0, \tag{111}$$

where D is admitted training tokens. Four parameter sizes at one token budget cannot identify both exponents; each size needs multiple budgets or a preregistered constrained fit. Functional metrics and contamination checks remain primary because perplexity scaling does not establish artifact correctness.

12.2 Masked, task-balanced next-token prediction

Let a serialized example contain target tokens $z_1, \ldots, z_T$, immutable context c, and nonnegative weights w_t. The normalized next-token loss is

$$\mathcal{L}_{\mathrm{NTP}} = -\frac{\sum_{t=1}^{T} w_t \log p_{\boldsymbol{\theta}}(z_t \mid z_{<t}, c)}{\sum_{t=1}^{T} w_t}, \qquad \sum_t w_t > 0. \tag{112}$$

Prompt, padding, retrieved evidence, tool output, and rejected-output regions receive $w_t = 0$. Approved artifacts and concise visible planning fields receive lane- and region-balanced weights. The denominator prevents long examples from dominating solely by token count.

For task lanes k sampled with target weights π_k, one implementation first computes a token-normalized example loss and then averages per task:

$$\mathcal{L}_{\mathrm{balanced}} = \sum_k \pi_k \frac{1}{|\mathcal{B}_k|} \sum_{e \in \mathcal{B}_k} \mathcal{L}_{\mathrm{NTP}}^{(e)}. \tag{113}$$

12.3 Verified supervised fine-tuning and distillation

For artifact regions $r \in \mathcal{R}_{\mathrm{region}}$, define each $\mathcal{L}_r$ as a token-normalized NLL. The supervised objective is

$$\mathcal{L}_{\mathrm{SFT}} = \sum_r \lambda_r \mathcal{L}_r, \qquad \lambda_r \geq 0, quad \sum_r \lambda_r = 1. \tag{114}$$

Regions include artifact text, bounded plan, evidence use, refusal/abstention, and diagnostic repair. Teacher rationales are not treated as executable evidence and private hidden chain-of-thought is not retained.

When teacher and student logits are available over the same allowed vocabulary, temperature-T_d distillation uses

$$\mathcal{L}_{\mathrm{KD}} = T_d^2 \sum_t D_{\mathrm{KL}}\left(p_{\mathrm{teacher}}^{(T_d)}(\cdot \mid z_{<t}, c) \,\middle\|\, p_{\boldsymbol{\theta}}^{(T_d)}(\cdot \mid z_{<t}, c)\right). \tag{115}$$

The T_d^2 factor compensates the gradient shrinkage introduced by softening. Every distilled artifact must still pass stronger deterministic checks than those visible to the teacher [40].

12.4 Preference optimization

Preference data is permitted only when both candidates already pass identical hard gates and differ in readability, maintainability, minimality, idiom, or visual clarity. For prompt x, preferred y_w, rejected y_l, policy $\pi_{\boldsymbol{\theta}}$, fixed reference π_{ref}, and temperature $\beta > 0$, direct preference optimization uses

$$\Delta_{\boldsymbol{\theta}}(x, y_w, y_l) = \log \frac{\pi_{\boldsymbol{\theta}}(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \log \frac{\pi_{\boldsymbol{\theta}}(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}, \tag{116}$$

$$\mathcal{L}_{\mathrm{DPO}} = -\mathbb{E}_{(x, y_w, y_l)}\left[\log \mathrm{sigmoid}(\beta \Delta_{\boldsymbol{\theta}}(x, y_w, y_l))\right]. \tag{117}$$

Sequence log probabilities are sums of token log probabilities under the same grammar support. Preference optimization cannot promote an incorrect candidate over a correct one [42].

12.5 Reinforcement learning with verifiable rewards

An additive reward can accidentally allow soft quality to rescue a schema or policy failure. HTT therefore defines a hard indicator $H(y,x)$ as in equation (2) and bounded utility $U(y,x) \in [0,1]$. A scalar representation that preserves the lexicographic order is

$$R(y,x) = \begin{cases} -1 - C_{\text{fail}}(y,x), & H(y,x) = 0, \\ 1 + U(y,x) - \lambda_c C_{\text{cost}}(y,x), & H(y,x) = 1, \end{cases} \tag{118}$$

where $C_{\text{fail}} \geq 0$ and the accepted-branch cost is bounded so $\lambda_c C_{\text{cost}} < 1$. Every feasible artifact then outranks every infeasible artifact. Backtest profit is excluded because it is unstable, exploitable, and regime dependent.

For a group of G_{group} candidates with rewards R_i, define sample mean and stabilized advantage

$$\bar{R} = \frac{1}{G_{\text{group}}} \sum_i R_i, \tag{119}$$

$$s_R = \sqrt{\frac{1}{G_{\text{group}} - 1} \sum_i (R_i - \bar{R})^2}, \tag{120}$$

$$A_i = \frac{R_i - \bar{R}}{s_R + \epsilon_R}. \tag{121}$$

For token ratio $\rho_{it} = \pi_{\boldsymbol{\theta}}(z_{it} \mid z_{i,<t}, x)/\pi_{\text{old}}(z_{it} \mid z_{i,<t}, x)$, a clipped group-relative objective is

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{i,t}\left[\min\left(\rho_{it} A_i, \text{clip}(\rho_{it}, 1-\epsilon_c, 1+\epsilon_c) A_i\right) - \beta_{\text{KL}} D_{\text{KL}}(\pi_{\boldsymbol{\theta}} \| \pi_{\text{ref}})\right]. \tag{122}$$

If the group variance is zero, advantages are set to zero rather than amplified by a tiny denominator. For leave-one-out baselines,

$$b_i = \frac{1}{G_{\text{group}} - 1} \sum_{j \neq i} R_j, \qquad A_i = R_i - b_i. \tag{123}$$

GRPO, RLOO, and ordinary supervised learning are controlled alternatives, not assumed winners [14, 15].

12.6 Multi-token prediction

For H auxiliary future-token heads, bounded indices, and nonnegative weights λ_j,

$$\mathcal{L}_{\text{MTP}} = -\sum_{j=1}^{H} \frac{\lambda_j}{T-j} \sum_{t=1}^{T-j} \log p_{\boldsymbol{\theta},j}(z_{t+j} \mid z_{\leq t}, c). \tag{124}$$

Auxiliary-head parameters, training FLOPs, and removal or reuse at inference must be reported. Published gains are scale dependent; HTT retains next-token prediction unless 450M- and 900M-parameter pilots show a functional improvement without reducing trunk depth [10].

13 Grammar-constrained generation and candidate selection

13.1 Normalized grammar-masked distribution

At decoding step t, let logits be $z_{t,v}$ and let the deterministic incremental parser expose allowed token set $\mathcal{A}_t$. The constrained distribution is

$$p_{\text{mask}}(v \mid y_{<t}, x) = \frac{\exp(z_{t,v}) \mathbb{1}[v \in \mathcal{A}_t]}{\sum_{u \in \mathcal{A}_t} \exp(z_{t,u})}. \tag{125}$$

Every disallowed token has exactly zero probability, and the denominator renormalizes over valid continuations. If $\mathcal{A}_t = \varnothing$, generation terminates as a parser failure rather than falling back to an unconstrained vocabulary. Grammar masking guarantees syntax only; types, capabilities, semantics, and behavior remain external gates [22].

13.2 Hard-feasible candidate selection

For candidate i, define length-normalized model log likelihood

$$\bar{\ell}_i = \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log p_\theta(y_{it} \mid y_{i,<t}, x). \tag{126}$$

Let z_i contain $\bar{\ell}_i$, compiler diagnostics, test coverage, mutation score, visual metrics, retrieval support, cost, and a learned verifier output. Continuous features are standardized with calibration-set means and scales. A linear reference selector is

$$s_i = w^\top \tilde{z}_i. \tag{127}$$

Selection occurs only inside the hard-feasible set:

$$\hat{i}_j = \arg\max_{i \in \mathcal{F}_j} s_{ji}. \tag{128}$$

If the set is empty, the request abstains or enters bounded repair. Execution-aware learned verification can improve ranking in prior code-generation settings, but HTT requires an independently trained verifier, near-miss mutants, calibration, and sealed tests [23, 24].

13.3 Calibration after selection

Selecting the maximum of K_{cand} candidates changes the score distribution. Calibration therefore occurs at request level *after* equation (128), separately for each serving candidate budget and quantization profile. Let $q_j \in [0,1]$ be calibrated probability that selected artifact $\hat{i}_j$ passes the sealed-equivalent outcome, and let $Y_j \in \{0,1\}$ be that outcome. For threshold τ,

$$\mathcal{I}_\tau = \{j : q_j \geq \tau\}, \tag{129}$$

$$\widehat{\text{Coverage}}(\tau) = \frac{|\mathcal{I}_\tau|}{N_{\text{eval}}}, \tag{130}$$

$$\widehat{\text{Risk}}(\tau) = \frac{\sum_{j \in \mathcal{I}_\tau} (1 - Y_j)}{|\mathcal{I}_\tau|}, \qquad |\mathcal{I}_\tau| > 0. \tag{131}$$

Risk is undefined at zero coverage. The threshold is chosen on calibration data using an upper confidence bound, not the point estimate alone. If e_τ failures occur among n_τ accepted requests, a one-sided Clopper–Pearson upper bound is

$$U_\alpha(e_\tau, n_\tau) = F^{-1}_{\text{Beta}}(1 - \alpha; e_\tau + 1, n_\tau - e_\tau), \tag{132}$$

with the conventional boundary case for $e_\tau = n_\tau$. Choose the lowest threshold satisfying $U_\alpha \leq r_{\max}$ and a minimum coverage constraint. This protects against accepting a tiny, apparently perfect sample.

Calibration diagnostics include Brier score

$$\text{Brier} = \frac{1}{N_{\text{eval}}} \sum_{j=1}^{N_{\text{eval}}} (q_j - Y_j)^2, \tag{133}$$

negative log likelihood

$$\text{NLL} = -\frac{1}{N_{\text{eval}}} \sum_j \left[Y_j \log q_j + (1 - Y_j) \log(1 - q_j) \right], \tag{134}$$

and bin-based expected calibration error

$$\text{ECE} = \sum_{b=1}^{B_{\text{bin}}} \frac{|I_b|}{N_{\text{eval}}} \left| \frac{1}{|I_b|} \sum_{j \in I_b} Y_j - \frac{1}{|I_b|} \sum_{j \in I_b} q_j \right|. \tag{135}$$

Because ECE depends on bins and can hide local error, reliability plots, Brier score, NLL, and risk–coverage curves are all reported [29].

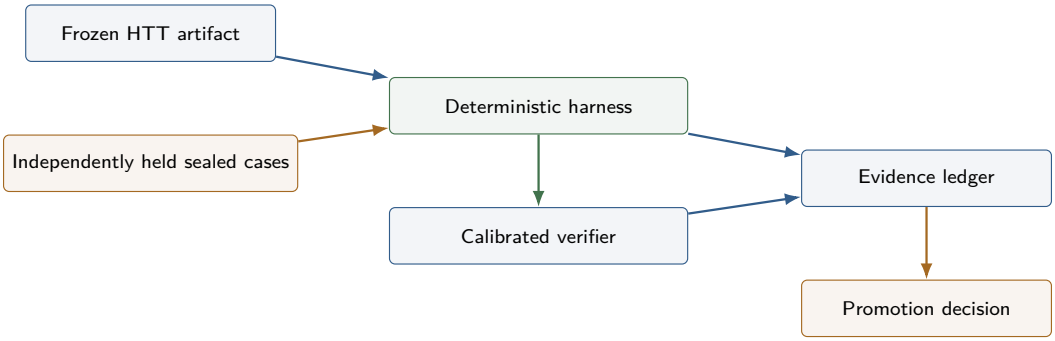


Figure 4: **Independent evidence path.** The model, trainer, and coordinator cannot write sealed labels or modify promotion policy. Evidence informs a later human decision.

14 Verification and evaluation architecture

14.1 Hard strategy and visualizer gates

Every strategy candidate must satisfy closed schema, exact-hash recomputation, parse and compile against the pinned SDK, namespace/type/member policy, isolated resource limits, deterministic tests, edge cases, metamorphic tests, mutation tests, and prohibited-capability scans. Every visualizer candidate must satisfy schema, allowlisted data sources and transforms, bounded geometry and update cost, deterministic rendering, screenshot/geometry comparison, accessibility, interaction replay, fallback behavior, and prohibited-capability scans. A learned verifier is supplementary evidence, not a substitute for these checks.

14.2 Primary request-level metrics

Let $A_j \in \{0,1\}$ indicate that request j is accepted after selection and calibration, and let $Y_j \in \{0,1\}$ denote functional correctness of its selected artifact. Three quantities must be reported together:

$$\widehat{\text{coverage}} = \frac{1}{N_{\text{eval}}} \sum_j A_j, \tag{136}$$

$$\widehat{\text{selective accuracy}} = \frac{\sum_j A_j Y_j}{\sum_j A_j}, \quad \sum_j A_j > 0, \tag{137}$$

$$\widehat{\text{accepted yield}} = \frac{1}{N_{\text{eval}}} \sum_j A_j Y_j. \tag{138}$$

The third equals coverage times selective accuracy. Reporting selective accuracy without coverage can reward a system that abstains on almost everything; reporting yield without selective risk can hide unsafe acceptance.

Compile-at-one is the fraction whose first generated candidate compiles without repair. Sealed functional pass-at-one is the fraction whose final deterministic-selection output passes the complete hidden suite under a fixed attempt budget. Candidate count, tool-call budget, context, evidence, and selection policy are part of the evaluated system and cannot differ across models without being reported.

14.3 Pass-at-k derivation

Suppose n sampled candidates contain c correct candidates and we choose $k \le n$ without replacement. There are $\binom{n}{k}$ equally sized subsets. The subsets containing no correct candidate choose all k

elements from the $n - c$ incorrect candidates, so

$$\mathbb{P}(\text{zero correct in } k) = \frac{\binom{n-c}{k}}{\binom{n}{k}}. \tag{139}$$

Taking the complement gives the standard estimator

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \tag{140}$$

It assumes exchangeable samples from a fixed generation policy. HTT's primary metric remains pass-at-one after a fixed deterministic selector; pass-at-k is diagnostic because larger sampling budgets spend more compute [56].

14.4 Perplexity and tokenizer comparability

For normalized natural-log NLL $\overline{\mathcal{L}}$,

$$\text{PPL} = \exp(\overline{\mathcal{L}}). \tag{141}$$

Per-token perplexity is not directly comparable across tokenizers because the atomic events differ. Tokenizer studies therefore also report NLL per byte or character and functional artifact metrics.

14.5 Paired significance and power

For two systems A and B evaluated on identical requests, let n_{10} count cases where only A succeeds and n_{01} cases where only B succeeds. The estimated paired difference is

$$\widehat{\Delta} = \frac{n_{10} - n_{01}}{N_{\text{eval}}}. \tag{142}$$

Under the null that discordant outcomes are equally likely, exact McNemar testing treats $n_{10} \mid (n_{10}+n_{01})$ as $\text{Binomial}(n_{10}+n_{01}, 1/2)$. When discordant counts are sufficiently large, the continuity-corrected approximation is

$$\chi^2 = \frac{(|n_{10} - n_{01}| - 1)^2}{n_{10} + n_{01}}. \tag{143}$$

Power depends on discordance, not only total accuracy. Let $p_{10} = \mathbb{P}(A = 1, B = 0)$, $p_{01} = \mathbb{P}(A = 0, B = 1)$, and $\Delta_* = p_{10} - p_{01}$ be the smallest meaningful paired improvement. A normal-approximation planning equation is

$$N_{\text{eval}} \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \left(p_{10} + p_{01} - \Delta_*^2\right)}{\Delta_*^2}. \tag{144}$$

The preregistration must supply plausible discordance from pilot data, desired power $1 - \beta$, and multiplicity correction. "At least three seeds" does not create a reliable seed-level confidence interval; examples are sampled across sufficient independent training runs or modeled hierarchically.

Repository families, synthetic lineages, and near-duplicate prompts violate request-level independence. Paired bootstrap resampling therefore operates at the highest independent repository/lineage cluster. Architecture comparisons use Holm correction or reserve an untouched confirmation set. Efficiency comparisons require identical hardware, precision, runtime, batch policy, context, output length, and thermal state.

14.6 Contamination and future evaluation

Training, teacher, retrieval, and generated corpora are compared with evaluation material using exact hashes, token similarity, AST and graph fingerprints, prompt lineage, and semantic clusters. A rolling future suite contains SDK and schema tasks acquired after the training cutoff. Public rolling suites eventually become trainable and cannot remain sealed forever; LiveCodeBench motivates the post-cutoff pattern but does not replace private Terminal cases [30].

15 Inference and serving mathematics

15.1 Artifact identity and request path

A served artifact identity binds weights, tokenizer, active task lane, quantization, runtime, kernels, context policy, retrieval pack, grammar, verifier, calibration map, and threshold. Any change creates a new evaluated artifact. The request path is

$$\begin{aligned}\text{authenticate} \to \text{authorize} \to \text{bind purpose} \to \text{retrieve} \to \text{generate} \to \text{gate} \\ \to \text{select} \to \text{calibrate} \to \text{inert artifact.}\end{aligned} \tag{145}$$

15.2 Quantization model and non-inferiority

For affine b-bit quantization with scale $s > 0$, zero point z, integer range $[q_{\min}, q_{\max}]$, and weight w,

$$q(w) = \operatorname{clip}(\operatorname{round}(w/s) + z, q_{\min}, q_{\max}), \tag{146}$$

$$\widehat{w} = s(q(w) - z). \tag{147}$$

When no clipping occurs, rounding gives $|w - \widehat{w}| \le s/2$. Network error can still compound through layers and grammar-boundary logits, so a per-weight bound does not establish artifact parity. Weight-only INT8, weight-only W4A16 AWQ/GPTQ, and W8A8 profiles are separate artifacts [43, 44, 45].

For paired functional outcomes, let $\Delta_q = p_{\text{quant}} - p_{\text{BF16}}$ and choose a non-inferiority margin $\delta > 0$. Quantization passes only if the one-sided lower confidence bound satisfies

$$\operatorname{LCB}_{1-\alpha}(\Delta_q) > -\delta, \tag{148}$$

while every hard gate, calibration ceiling, coverage floor, and slice-specific safety metric also passes. Average non-inferiority cannot compensate for a prohibited-capability regression.

15.3 Continuous batching and cache isolation

Batching is allowed only among compatible artifact identities and privacy domains. Prefix reuse requires identical tenant authorization, evidence hash, task lane, model, grammar, and policy. User or private-evidence prefixes are never shared across unauthorized principals. Released cache pages are zeroized according to the serving threat model. The capacity constraint for live sequences j is

$$\sum_j M_{\text{KV,hybrid}}(B_j, N_j) + M_{\text{weights}} + M_{\text{workspace}} + M_{\text{activations}} \le M_{\text{device,reserved}}, \tag{149}$$

with an explicit safety reserve rather than optimistic use of all device memory.

15.4 Speculative decoding

Let draft distribution $q(x)$ and target distribution $p(x)$ have identical grammar support. A draft token $x \sim q$ is accepted with

$$a(x) = \min\left(1, \frac{p(x)}{q(x)}\right). \tag{150}$$

On rejection, sample from residual distribution

$$r(x) = \frac{(p(x) - q(x))_+}{\sum_u (p(u) - q(u))_+}. \tag{151}$$

These rules preserve the target distribution; greedy acceptance does not. Speculation is optional and must retain identical grammar masks, deterministic tool boundaries, and artifact parity [27].

15.5 Operational gates, not performance claims

The initial qualification records p50/p95 time to first token, tokens per second, tool latency, request latency, peak device and host memory, power, and energy per accepted artifact at 8K, 16K, and 32K contexts. Gates include no out-of-memory at admitted concurrency, no hard-gate regression, request-level calibration within its ceiling, deterministic replay, and clean rollback. No number in this protocol claims that those gates have been met.

16 Ablation design and parameter matching

An ablation changes one hypothesis while holding data order, token budget, optimizer, evaluation, context, candidate/tool budget, and hardware constant. Raw-shape comparisons and parameter-matched comparisons answer different questions and both are reported.

Changing h_{kv} from 4 to 16 at fixed d, d_h adds per block

$$\Delta P_{\mathrm{MHA,block}} = 2d(16-4)d_h \tag{152}$$

$$= 2(2{,}048)(12)(128) = 6{,}291{,}456, \tag{153}$$

or

$$\Delta P_{\mathrm{MHA,total}} = 32(6{,}291{,}456) = 201{,}326{,}592. \tag{154}$$

Reducing SwiGLU width from 5,632 to 4,608 removes exactly $3d(5{,}632-4{,}608) = 6{,}291{,}456$ parameters per block, giving a parameter-matched MHA control. The raw control measures the direct engineering replacement; the compensated control separates head sharing from total capacity.

Table 9: Mandatory architecture ablations.

ID	Change	Control	Primary decision signal
A1	all-global attention	3:1 hybrid	functional accuracy versus memory/latency
A2	5:1 local/global	3:1 hybrid	efficiency without long-context regression
A3	2K and 8K local windows	4K	context-slice correctness and cache
A4	1, 2, 8, 16 KV heads	4	quality, cache, and compensated capacity
A5	remove task lanes	rank-64 lanes	interference by artifact family
A6	rank 32 and 128	rank 64	accuracy per active parameter
A7	remove parser embedding	optional table	value beyond mandatory grammar mask
A8	16K, 32K, 48K vocabulary	32K	tokens, exact copy, quality, embedding size
A9	24 and 36 blocks	32	depth at matched active parameters
A10	QK-Norm off	QK-Norm on	stability and functional quality

Deferred challengers include mixture of experts, dynamic depth, state-space hybrids, byte-latent tokenization, ternary weights, and native vision. They enter only after the dense reference is reproducible [9, 47, 37, 38].

17 Security, privacy, and failure analysis

Table 10: Principal failure modes and controls.

Failure	Why it matters	Required control
Failure	Why it matters	Required control
Hallucinated API	compiles against no real SDK	versioned retrieval, exact compile, abstention
Semantically wrong strategy	syntax is insufficient	properties, metamorphic and mutation tests
Prompt injection in evidence	retrieved text may claim authority	provenance, instruction/data separation, quarantine
Prohibited capability	generated source exceeds scope	closed schemas, compiler/IL scan, sandbox
Authority-field fabrication	model attempts self-approval	coordinator recomputes and rejects fields
Verifier reward hacking	score exceeds real quality	independent hidden tests, near-miss mutants, rotation
Cross-request leakage	private context reaches another user	cache isolation, zeroization, canaries
Training contamination	sealed answers become memorized	lineage graph, time split, independent custody
Quantization drift	boundary logits or behavior change	paired non-inferiority and hard-gate parity
Distribution shift	SDK or conventions change	future suite, retrieval versioning, rollback

Privacy testing uses approved synthetic canaries for exact and approximate extraction, membership inference, cross-request leakage, prefix-cache isolation, log redaction, retention, deletion, teacher exposure, and quantized-artifact parity. Real secrets are never used as probes. The model card must state that the system is purpose-limited, cannot predict markets, has no trading authority, may abstain, requires deterministic validation, and has not been shown correct until sealed evidence exists.

18 Implementation roadmap and decision gates

Gate 0 — protocol review.
Review purpose, artifact boundaries, mathematical assumptions, corpus rights, architecture candidates, target hardware, evaluation ownership, and budget.
Gate 1 — data and tokenizer readiness.
Require an audited corpus manifest, split-isolation report, tokenizer study, executable harnesses, and preregistered metrics.
Gate 2 — small-pilot selection.
Train at least three parameter-matched candidates at each of several sizes and token budgets; select using functional and efficiency evidence, not perplexity alone.
Gate 3 — scaling authorization.
Fit scaling curves, profile achieved utilization, rehearse checkpoint recovery, and approve the full-run budget separately.
Gate 4 — frozen-model evaluation.
Freeze weights, tokenizer, runtime, quantization, retrieval, grammar, verifier, calibration map, and thresholds before an independent operator unseals results.
Gate 5 — promotion and shadow serving.
Require signed evidence, no hard-gate regression, accepted risk, rollback, startup attestation, and inert shadow artifacts.

Gate 6 — constrained rollout.
Begin with low-risk mock/research use, strict capacity, logging, and human review; expand only from monitored evidence.

19 Limitations and falsifiability

The paper describes an architecture and protocol, not a trained model. A narrow 1.5B model may underfit compositional language, overfit templates, or require more clean tokens than the rights-approved domain corpus can supply. Scratch training avoids inherited data uncertainty but may be worse than continued pretraining from an eligible small code model; both must be compared under the same retrieval, tools, and evaluation [39]. Long mathematical derivations do not establish kernel performance, generalization, or safety. Grammar masks establish syntax, not meaning. Deterministic tests are only as strong as their coverage. Verifiers can be miscalibrated or reward-hacked. Human approval can fail operationally.

The proposal is rejected or revised if a simpler parameter-matched baseline wins sealed functional pass-at-one, if the narrow corpus cannot support future-version generalization, if quantization breaks hard gates, if request-level risk cannot be bounded at useful coverage, or if latency/memory targets cannot be met on intended hardware. Architectural novelty is not a promotion criterion.

20 Conclusion

HTT-1.5B-r0 is a conservative transformer embedded in an aggressive verification boundary. Stable skills enter weights; mutable private facts enter signed retrieval; syntax is restricted by a parser; semantics are judged by compilers, renderers, and tests; uncertainty is handled after candidate selection; and release remains external. The proposed dimensions imply a 1.523B core stored model, a 1.524B parser-enabled model, and a 0.6875 GiB batch-one BF16 KV cache at 32K under the stated hybrid schedule. The derivations make those claims auditable and distinguish causal prefill from decode, weight memory from training state, and score-position savings from total runtime.

The next research deliverables are a governed corpus manifest, tokenizer report, executable sealed harness, small-model scaling and ablation report, and calibrated request-level selector. Only those experiments can determine whether the architecture is accurate, fast, parameter-efficient, and safely narrow enough for Terminal strategy and visualizer authoring.

A Additional derivations and dimensional checks

A.1 Leading-order parameter expression

Ignoring small normalization, identity, adapter, and parser terms, a tied-embedding GQA/SwiGLU model has

$$P_{\text{lead}} = |\mathcal{V}|d + n_{\text{layer}}\left[2d^2 + 2dh_{\text{kv}}d_h + 3dd_{\text{ff}}\right]. \tag{155}$$

If $h_q d_h = d$ and $d_{\text{ff}} \approx 8d/3$,

$$P_{\text{lead}} \approx |\mathcal{V}|d + n_{\text{layer}}d^2\left(10 + 2\frac{h_{\text{kv}}}{h_q}\right). \tag{156}$$

MHA has $h_{\text{kv}}/h_q = 1$ and coefficient 12; 4:1 GQA has ratio $1/4$ and coefficient 10.5. This expression is useful for matched-shape reasoning but is not the executable parameter total in table 4.

A.2 Units audit

In equation (87), parameters multiplied by bytes per parameter yield bytes. In equation (89), the factors are key/value copies, batch, layers, tokens, heads, elements per head, and bytes per element,

again yielding bytes. In equation (94), parameters and tokens are dimensionless counts and the coefficient carries approximate floating-point operations per parameter-token. In equation (106), every utility term is standardized and therefore dimensionless while the constraint is measured in tokens. These checks prevent adding quantities with incompatible scales.

B Proposed visualizer-IR validation rules

The proposed visualizer artifact obeys the following minimum rules:

1. `schemaVersion` is exactly `hyperion-terminal-visualizer/v1`.
2. Every pane references a declared, allowlisted data source.
3. Every signal is an allowlisted transform with finite cost and explicit null behavior.
4. Every interaction has bounded, deterministic behavior and no arbitrary callback.
5. Rendering rejects zero-area panes, duplicate identifiers, invalid scales, and unsupported update rates.
6. Style values come from bounded enums/ranges or approved theme tokens.
7. Network, filesystem, process, reflection, package, native-code, broker, and credential concepts are unrepresentable.
8. The compiler emits only an approved renderer and records the source schema, compiler version, and output hash.
9. Screenshot, geometry, interaction, accessibility, and performance checks must pass before human review.

C Minimum sealed-suite composition

The final model-selection set includes, without reusing calibration cases:

- held-out strategy DSL, C#, numeric, and state-machine problems;
- held-out visualizer grammar and accessibility problems;
- compile and runtime failures requiring exact repair;
- current, stale, missing, contradictory, and adversarial evidence;
- prohibited capability and injection tests;
- 8K, 16K, 32K, and short-context regression slices;
- byte, Unicode, numeric, locale, and unusual-identifier cases;
- future SDK/schema tasks acquired after corpus closure; and
- enough independent repository/lineage clusters to meet the preregistered paired-power plan.

D Reproducibility checklist

- Freeze source revision, container/compiler, tokenizer, data manifest, data order, optimizer, scheduler, random-number states, kernels, and hardware topology.
- Record executable parameter counts rather than relying only on analytic estimates.
- Record per-slice losses and functional metrics at every checkpoint.
- Keep development, calibration, sealed, and future custody separate.
- Bind every result to candidate count, tool budget, context, retrieval pack, and verifier.
- Publish negative and null ablations, not only the winning configuration.
- Rebuild the paper from this LaTeX source and retain compiler logs with the submission bundle.

References

[1] A. Vaswani et al. “Attention Is All You Need.” *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762.

[2] B. Zhang and R. Sennrich. “Root Mean Square Layer Normalization.” *NeurIPS*, 2019. https://arxiv.org/abs/1910.07467.

[3] N. Shazeer. “GLU Variants Improve Transformer.” 2020. https://arxiv.org/abs/2002.05202.

[4] J. Su et al. “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021. https://arxiv.org/abs/2104.09864.

[5] J. Ainslie et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.” *EMNLP*, 2023. https://arxiv.org/abs/2305.13245.

[6] T. Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” *NeurIPS*, 2022. https://arxiv.org/abs/2205.14135.

[7] T. Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.” 2023. https://arxiv.org/abs/2307.08691.

[8] A. Q. Jiang et al. “Mistral 7B.” 2023. https://arxiv.org/abs/2310.06825.

[9] D. Raposo et al. “Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models.” 2024. https://arxiv.org/abs/2404.02258.

[10] F. Gloeckle et al. “Better & Faster Large Language Models via Multi-token Prediction.” 2024. https://arxiv.org/abs/2404.19737.

[11] B. Hui et al. “Qwen2.5-Coder Technical Report.” 2024. https://arxiv.org/abs/2409.12186.

[12] D. Guo et al. “DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence.” 2024. https://arxiv.org/abs/2401.14196.

[13] P. Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS*, 2020. https://arxiv.org/abs/2005.11401.

[14] A. Ahmadian et al. “Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs.” 2024. https://arxiv.org/abs/2402.14740.

[15] Z. Shao et al. “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.” 2024. https://arxiv.org/abs/2402.03300.

[16] S. Zhang et al. “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation.” 2023. https://arxiv.org/abs/2303.12570.

[17] D. Wu et al. “Repoformer: Selective Retrieval for Repository-Level Code Completion.” *ICML*, 2024. https://arxiv.org/abs/2403.10059.

[18] T. Zhang et al. “RAFT: Adapting Language Model to Domain Specific RAG.” 2024. https://arxiv.org/abs/2403.10131.

[19] S. M. Xie et al. “DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining.” *NeurIPS*, 2023. https://arxiv.org/abs/2305.10429.

[20] K. Lee et al. “Deduplicating Training Data Makes Language Models Better.” *ACL*, 2022. https://arxiv.org/abs/2107.06499.

[21] A. Lozhkov et al. “StarCoder 2 and The Stack v2: The Next Generation.” 2024. https://arxiv.org/abs/2402.19173.

[22] S. Ugare et al. “SynCode: LLM Generation with Grammar Augmentation.” 2024. https://arxiv.org/abs/2403.01632.

[23] A. Ni et al. “LEVER: Learning to Verify Language-to-Code Generation with Execution.” *ICML*, 2023. https://arxiv.org/abs/2302.08468.

[24] K. Cobbe et al. “Training Verifiers to Solve Math Word Problems.” 2021. https://arxiv.org/abs/2110.14168.

[25] H. Lightman et al. “Let’s Verify Step by Step.” *ICLR*, 2024. https://arxiv.org/abs/2305.20050.

[26] W. Kwon et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention.” *SOSP*, 2023. https://arxiv.org/abs/2309.06180.

[27] Y. Leviathan et al. “Fast Inference from Transformers via Speculative Decoding.” *ICML*, 2023. https://arxiv.org/abs/2211.17192.

[28] J. Liu et al. “Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation.” *NeurIPS*, 2023. https://arxiv.org/abs/2305.01210.

[29] S. Kadavath et al. “Language Models (Mostly) Know What They Know.” 2022. https://arxiv.org/abs/2207.05221.

[30] N. Jain et al. “LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.” 2024. https://arxiv.org/abs/2403.07974.

[31] S. Gunasekar et al. “Textbooks Are All You Need.” 2023. https://arxiv.org/abs/2306.11644.

[32] M. Abdin et al. “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone.” 2024. https://arxiv.org/abs/2404.14219.

[33] J. Kaplan et al. “Scaling Laws for Neural Language Models.” 2020. https://arxiv.org/abs/2001.08361.

[34] J. Hoffmann et al. “Training Compute-Optimal Large Language Models.” *NeurIPS*, 2022. https://arxiv.org/abs/2203.15556.

[35] A. Yang et al. “Qwen3 Technical Report.” 2025. https://arxiv.org/abs/2505.09388.

[36] Gemma Team. “Gemma 3 Technical Report.” 2025. https://arxiv.org/abs/2503.19786.

[37] A. Pagnoni et al. “Byte Latent Transformer: Patches Scale Better Than Tokens.” 2024. https://arxiv.org/abs/2412.09871.

[38] S. Ma et al. “The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits.” 2024. https://arxiv.org/abs/2402.17764.

[39] N. Pinnaparaju et al. “Stable Code Technical Report.” 2024. https://arxiv.org/abs/2404.01226.

[40] C.-Y. Hsieh et al. “Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes.” *ACL*, 2023. https://arxiv.org/abs/2305.02301.

[41] A. Singh et al. “Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models.” *TMLR*, 2024. https://arxiv.org/abs/2312.06585.

[42] R. Rafailov et al. “Direct Preference Optimization: Your Language Model is Secretly a Reward Model.” *NeurIPS*, 2023. https://arxiv.org/abs/2305.18290.

[43] E. Frantar et al. “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.” *ICLR*, 2023. https://arxiv.org/abs/2210.17323.

[44] J. Lin et al. “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.” *MLSys*, 2024. https://arxiv.org/abs/2306.00978.

[45] G. Xiao et al. “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.” *ICML*, 2023. https://arxiv.org/abs/2211.10438.

[46] A. Gu and T. Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces.” 2023. https://arxiv.org/abs/2312.00752.

[47] T. Dao and A. Gu. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality.” *ICML*, 2024. https://arxiv.org/abs/2405.21060.

[48] S. Yang et al. “Parallelizing Linear Transformers with the Delta Rule over Sequence Length.” *NeurIPS*, 2024. https://arxiv.org/abs/2406.06484.

[49] N. Shazeer et al. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.” *ICLR*, 2017. https://arxiv.org/abs/1701.06538.

[50] W. Fedus et al. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.” *JMLR*, 2022. https://arxiv.org/abs/2101.03961.

[51] L. B. Allal et al. “SantaCoder: Don’t Reach for the Stars!” 2023. https://arxiv.org/abs/2301.03988.

[52] CodeGemma Team. “CodeGemma: Open Code Models Based on Gemma.” 2024. https://arxiv.org/abs/2406.11409.

[53] M. Bavarian et al. “Efficient Training of Language Models to Fill in the Middle.” 2022. https://arxiv.org/abs/2207.14255.

[54] Z. Liu et al. “MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases.” 2024. https://arxiv.org/abs/2402.14905.

[55] B. Chen et al. “CodeT: Code Generation with Generated Tests.” *ICLR*, 2023. https://arxiv.org/abs/2207.10397.

[56] M. Chen et al. “Evaluating Large Language Models Trained on Code.” 2021. https://arxiv.org/abs/2107.03374.

Repository sources consulted

The following repository sources define implemented boundaries and are not empirical evidence that HTT has been trained or evaluated:

- `AGENTS.md` and `README.md`;
- `docs/architecture/hyperion-frontier-architecture.md`;
- `docs/engineering/hyperion-llm-coordinator-handbook.md`;
- `docs/guides/hyperion-coordinator.md`;
- `docs/research/kimi-k3-adoption-notes.md`;
- `src/windows/AI/TradingTerminal.Ai.Coordinator.Contracts/Contracts/CoordinatorStrategyArtifacts.cs`;
- `src/windows/Tools/DaxAlgo.Daxq.Compiler/Authoring/DaxqAuthoringContracts.cs`; and
- read-only shared-core chart and strategy-builder documentation under `public/`.